Persistência de dados

Programação para a Web 2

Aula 14

Evandro J.R. Silva

Por que o *backend* precisa persistir dados

- O pedido acaba, mas o fato não
  - HTTP é *stateless*.
  - Cada mensagem que chega é interpretada e produz outra mensagem.
    - O processo que atendeu pode ser outro na próxima chamada, ou pode ter sido reiniciado, ou ainda nem existir mais.
    - Se o único lugar de uma conta que foi criada tenha sido uma variável de uma determinada requisição, essa conta deixa de existir com a resposta.

Por que o *backend* precisa persistir dados

- Persistir é tornar um fato recuperável por um processo que não participou da sua criação.
  - Não confundir com “usar banco de dados”.
  - O banco de dados é um meio, mas o requisito para persistência é a **durabilidade** do que o negócio trata como acontecido.

Por que o *backend* precisa persistir dados

- Nem todo estado é persistência.
  - Não podemos confundir “o *backend* lida com dados” com “tudo deve ser gravado”.
  - Existem três tipos básicos de duração de um estado:
    - **Duração da requisição:** o estado existe até a resposta, ou seja, não persiste.
    - **Duração derivada, reconstruível:** o estado pode ser armazenado em memória (ou outro tipo de armazenamento rápido) e pode deixar de existir sem corromper o negócio. Ex.: uma página em cache, um contador de visitas aproximado, uma sessão que pode ser refeita com *login*.
    - **Duração de negócio:** se perdido, muda o que o sistema afirma ter acontecido, por exemplo, conta criada, transferência aceita, nota lançada. Este é o estado que a persistência precisa guardar.

Separação entre lógica de negócio e armazenamento

- Cada uma responde perguntas diferentes
  - Lógica de negócio
    - Este fato pode ocorrer?
    - O saldo cobre o valor?
    - Esta transição de pedido é legal?
    - O e-mail já cadastrado impede outra conta?
  - Armazenamento
    - Onde isto será gravado?
    - Como se relê?

Separação entre lógica de negócio e armazenamento

Do lado da regra	Do lado do armazenamento
Nomes de domínio: Conta, Pedido, Transferência	Tabela, coleção, chave, índice
Invariantes: saldo não negativo, transição legal	Como a invariante é imposta na gravação
Falhas nomeadas: `saldoInsuficiente`	Tradução de falha técnica: conexão, conflito, *timeout*
Decisão de aceitar ou recusar	Decisão de confirmar a gravação
Não conhece SQL, documento nem *host*	Não decide se a transferência é válida

Separação entre lógica de negócio e armazenamento

- Essa diferença pode ficar bem tênue
  - As regras de um sistema podem ter sido implementadas dentro dos comandos de gravação.
- Na prática, deve haver uma interface entre a regra e o armazenamento.
  - A regra pede “guarde isto” ou “traga aquilo”, mas sem dizer como.

```
regras:
    conta = contas.buscar(origem)
    se conta.saldo < valor -> SaldoInsuficiente
    conta.debitar(valor)
    contas.guardar(conta)

armazenamento:
    buscar(id) -> lê e reconstitui Conta
    guardar(c) -> grava o estado de c
    // não sabe o que é saldo suficiente
```

Separação entre lógica de negócio e armazenamento

- MAS
  - **Não apaga o modelo de dados**
    - A forma de gravar influencia o que pode ser lido com pouco custo. Ignorar isso e projetar a regra como se toda leitura fosse de graça empurra a complexidade para consultas impossíveis.
  - **Não empurra a transação para dentro da entidade**
    - Quem abre e fecha uma unidade de trabalho é a aplicação, e não o objeto.
  - **Não duplica validação sem critério**
    - A regra recusa saldo negativo. O armazenamento pode recusar também, como rede de segurança. As duas recusas precisam contar a mesma história: a de domínio volta como falha esperada; a falha técnica inesperada não vira regra.
  - **Não é camada por cerimônia**
    - A interface com um método que só repassa a consulta, e uma regra que não tem invariante, não separam nada. Separam quando há o que proteger da mudança de meio.

Tipos gerais de armazenamento

- A modelagem de dados é uma das partes mais importantes do desenvolvimento de *software*, porque ela tem um impacto profundo em como o *software* é escrito, como também em como pensamos no problema a ser resolvido.
- Além disso
  - **São uma ponte entre times diferentes**. Por exemplo, analistas de dados e equipe de desenvolvimento.
  - **Ajudam a manter a integridade dos dados**, através de restrições que previnem dados inválidos, incompletos ou duplicados.
  - **Têm impacto na performance**, pois bases de dados bem modelados permitem consultas rápidas, mesmo algumas mais complexas.
  - **Facilitam a escalabilidade**, ao antecipar o crescimento futuro e já estar preparado para novos requisitos.
  - **Mantêm a consistência dos dados**, de forma que eles tenham o mesmo significado nas diferentes partes de um sistema.

Tipos gerais de armazenamento

- Tipos principais
  - Modelos clássicos
    - Modelo hierárquico;
    - Modelo de rede (*network model*);
    - Modelo relacional.
  - Modelos modernos
    - Modelo orientado a objetos;
    - Modelo objeto-relacional;
    - NoSQL.
    - Modelo vetorial.

Tipos gerais de armazenamento

- Modelo Hierárquico
  - Organiza os dados em uma árvore, onde cada registro (nó/nodo) possui um único pai (exceto o nó raiz) e múltiplos filhos.
  - É um modelo antigo, datado dos primórdios da área de BD, utilizado nos primeiros DBMS de *mainframes*. Seus equivalentes modernos são armazenamento com XML/JSON.
  - Casos de uso: sistemas legados de banco, sistemas de reserva de voo.
  - Pró(s): consultas um-para-muitos bastante rápidas.
  - Contra(s): difícil de tratar relacionamentos muitos-para-muitos.

```
<Cliente id="1">
    <Nome>Fulano da Silva</Nome>
    <Pedidos>
        <Pedido id="1" data="2026-10-08"/>
    </Pedidos>
</Cliente>
```

Tipos gerais de armazenamento

- Modelo de Rede (*Network Model*)
  - É uma forma flexível de representar dados e relacionamentos, especialmente útil para conexões muitos-para-muitos complexa.
  - Utiliza uma estrutura de grafos com nodos (entidades) e arestas (relacionamentos) para organizar os dados, permitindo caminhos de acesso direto e mais eficientes em relação ao modelo hierárquico.
  - Pró(s): lida com relacionamentos muitos-para-muitos complexos.
  - Contra(s): é difícil de manter e de fazer consultas (*queries*).

Tipos gerais de armazenamento

- Modelo Relacional
  - É o mais conhecido. Estrutura dados em tabelas com linhas e colunas, permitindo armazenamento, consulta e gerenciamento de informação eficientes.
  - Pró(s): suporte de SQL, conformidade ao ACID, normalização.
  - Contra(s): *joins* podem ser computacionalmente caras em maiores escalas. *Impedance mismatch*, a disconexão entre o sistema orientado a objetos e o modelo relacional.

Tipos gerais de armazenamento

- Modelo Orientado a Objetos
  - Combina banco de dados e conceitos de programação orientada a objetos. Dá suporte a herança e encapsulamento.
- Modelo Objeto-Relacional (ORM)
  - Híbrido entre os modelos Relacional e Orientado a Objetos.

Tipos gerais de armazenamento

- Modelo NoSQL
  - O termo teve origem em uma *hashtag* no Twitter para o *Meetup on Open Source, Distributed, Nonrelational Databases* em 2009.
  - Não se refere a uma única tecnologia e, por causa disso, alguns interpretam a sigla retroativamente como *Not Only SQL*.
  - Seu surgimento e rápida adoção pode ser atribuída a alguns fatores
    - A necessidade de uma escalabilidade maior que os bancos de dados relacionais são capazes de alcançar, incluindo bases de dados muito grandes ou com uma taxa de escrita muito alta.
    - Uma ampla preferência por *software* livre e *open source*, em vez dos DBMS comerciais.
    - Operações de consulta (*queries*) especializadas que não possuem um bom suporte em modelos relacionais.
    - Frustração com restrições impostas nos *schemas* relacionais, e o desejo de utilizar modelos mais expressivos e dinâmicos.

Tipos gerais de armazenamento

- Modelo NoSQL
  - Possui alguns tipos principais
    - *Document model*
      - Armazena os dados em documentos flexíveis e auto-descritíveis, como JSON, BSON e XML. Cada documento possui uma estrutura única, fazendo com que esse modelo seja altamente adaptável a requisitos de dados que mudam.
      - Pró(s): *schemas* flexíveis que evoluem facilmente; eficiente para dados aninhados e hierárquicos.
      - Contra(s): consultas sobre vários documentos são complexas, e estruturas inconsistentes podem complicar a lógica de aplicação.
      - Exemplos: *Couchbase*, *MongoDB*.
    - *Key-value model*
    - *Columnar model*
    - *Graph model*
    - *Time series model*
    - *Multi-model*

Tipos gerais de armazenamento

- Modelo NoSQL
  - Possui alguns tipos principais
    - *Document model*
    - *Key-value model*
      - Organiza os dados em simples pares *key-value*. Cada chave identifica unicamente um trecho de dados, permitindo consultas rápidas.
      - Pró(s): latência ultra-baixa e alta escalabilidade; simples e eficiente para operações de leitura e escrita.
      - Contra(s): capacidades de consulta limitadas a acesso baseado em *key-value*; difícil para encaixar relacionamentos complexos ou dados estruturados.
      - Exemplos: *Redis*, *Amazon DynamoDB*.
    - *Columnar model*
    - *Graph model*
    - *Time series model*
    - *Multi-model*

Tipos gerais de armazenamento

- Modelo NoSQL
  - Possui alguns tipos principais
    - *Document model*
    - *Key-value model*
    - *Columnar model*
      - Armazena dados em colunas, em vez de linhas, permitindo o rápido a acesso a bases de dados grandes e armazenamento eficiente de dados esparsos.
      - Pró(s): otimizado para alta taxa de transferência de escrita e leitura; escala horizontalmente com facilidade.
      - Contra(s): a modelagem de relacionamentos um-para-muitos é complexa; padrões de consulta (*query patterns*) precisam ser planejadas cuidadosamente para performance ótima.
      - Exemplos: *Apache Cassandra*, *Apache HBase*.
    - *Graph model*
    - *Time series model*
    - *Multi-model*

Tipos gerais de armazenamento

- Modelo NoSQL
  - Possui alguns tipos principais
    - *Document model*
    - *Key-value model*
    - *Columnar model*
    - *Graph model*
      - Usa nodos (entidades) e arestas (relacionamentos) para representar e armazenar dados altamente conectados, facilitando a travessia de redes complexas.
      - Pró(s): eficiente no tratamento de relacionamentos profundos e complexos; suporta consultas rápidas em caminhos de relacionamento (*relationship paths*).
      - Contra(s): requer linguagens de consulta especializadas, como Cypher ou Gremlin; menos eficiente para armazenamento de dados planos em larga escala.
      - Exemplos: *Neo4j*, *Amazon Neptune*.
    - *Time series model*
    - *Multi-model*

Tipos gerais de armazenamento

- Modelo NoSQL
  - Possui alguns tipos principais
    - *Document model*
    - *Key-value model*
    - *Columnar model*
    - *Graph model*
    - *Time series model*
      - Especializado em armazenar e recuperar dados temporais como logs, eventos, e leituras de sensores.
      - Pró(s): otimizado para alta taxa de escrita e consultas baseadas em tempo (*time-based*); suporta expiração automática por data e compactação.
      - Contra(s): ideal somente para dados temporais; capacidades de consulta são limitadas a padrões baseados em tempo.
      - Exemplos: *InfluxDB*, *TimescaleDB*.
    - *Multi-model*

Tipos gerais de armazenamento

- Modelo NoSQL
  - Possui alguns tipos principais
    - *Document model*
    - *Key-value model*
    - *Columnar model*
    - *Graph model*
    - *Time series model*
    - *Multi-model*
      - Dão suporte a múltiplos modelos de dados NoSQL como documento, *key-value*, grafo e coluna em uma única *engine*, dando maior flexibilidade em como os dados são armazenados e consultados.
      - Pró(s): suporta diversos padrões de acesso de dados em uma única base de dados; reduz a necessidade de múltiplas tecnologias de base de dados; flexível para requisitos de aplicação em evolução.
      - Contra(s): pode introduzir *trade-offs* em performance para certos modelos; pode adicionar complexidade no *deploy* e configurações.
      - Exemplos: *Couchbase*, *ArangoDB*.

Tipos gerais de armazenamento

- Modelo Vetorial
  - Também conhecido como banco de dados de pesquisa vetorial ou mecanismo de pesquisa por similaridade vetorial, armazena, recupera e pesquisa por vetores.
  - Os dados ainda são tabulares, mas cada linha representa um ponto em um espaço hiperdimensional.
  - Bancos de dados vetoriais são ideais para aplicações que exigem correspondência rápida e precisa de dados com base na similaridade, em vez de valores exatos (e.g., LLMs, ou IAs Generativas).
  - Exemplos: *Qdrant*, *Milvus*, *Pinecone*, *pgvector for PostgreSQL*, *MongoDB Atlas Vector Search*, *Google Cloud BigQuery/AlloyDB*.